

Workshop No. 4 — Layered Architecture for a Domotic Circuit Simulator (Python)

Object-Oriented Programming
Semester 2025-II
Universidad Nacional de Colombia

Authors:

Andres David Hincapié Álvarez
Samuel Alejandro Bulla Fiquitiva
Daniel Alejandro Clavijo Gonzalez

Layered Design Review:

The domotic circuit simulator has been reorganized into a **three-layer architecture**, separating responsibilities to improve modularity, maintainability, and alignment with SOLID principles. The class diagrams and design documents have been updated accordingly.

Layer 1: Presentation Layer

Description:

In this design, the simulator does **not** include a graphical or textual User Interface class inside the core model. Any interface would belong to an external module, outside the scope of the workshop and the UML. This correction was made based on instructor feedback: *the controller and simulation logic should not include UI components*.

Layer 2: Logic Layer

Classes:

- **Circuit_Simulator**
- **Controller**
- **UserInterface**
- **Bulb**
- **Actuator**
- **Component**
- **Sensor**
- **Switch**
- **DataManager**

Responsibilities:

- Coordinate the overall simulation process
- Manage relationships between sensors and actuators
- Execute processing rules
- read and save .json archives
- Apply system logic independently of the data structures or presentation concerns

Interactions:

- CircuitSimulator manages collections of Sensor, Controller, and Actuator objects.
- Controller receives data from Sensors, processes logic, and sends signals to Actuators

Layer 3: Component Layer

Classes:

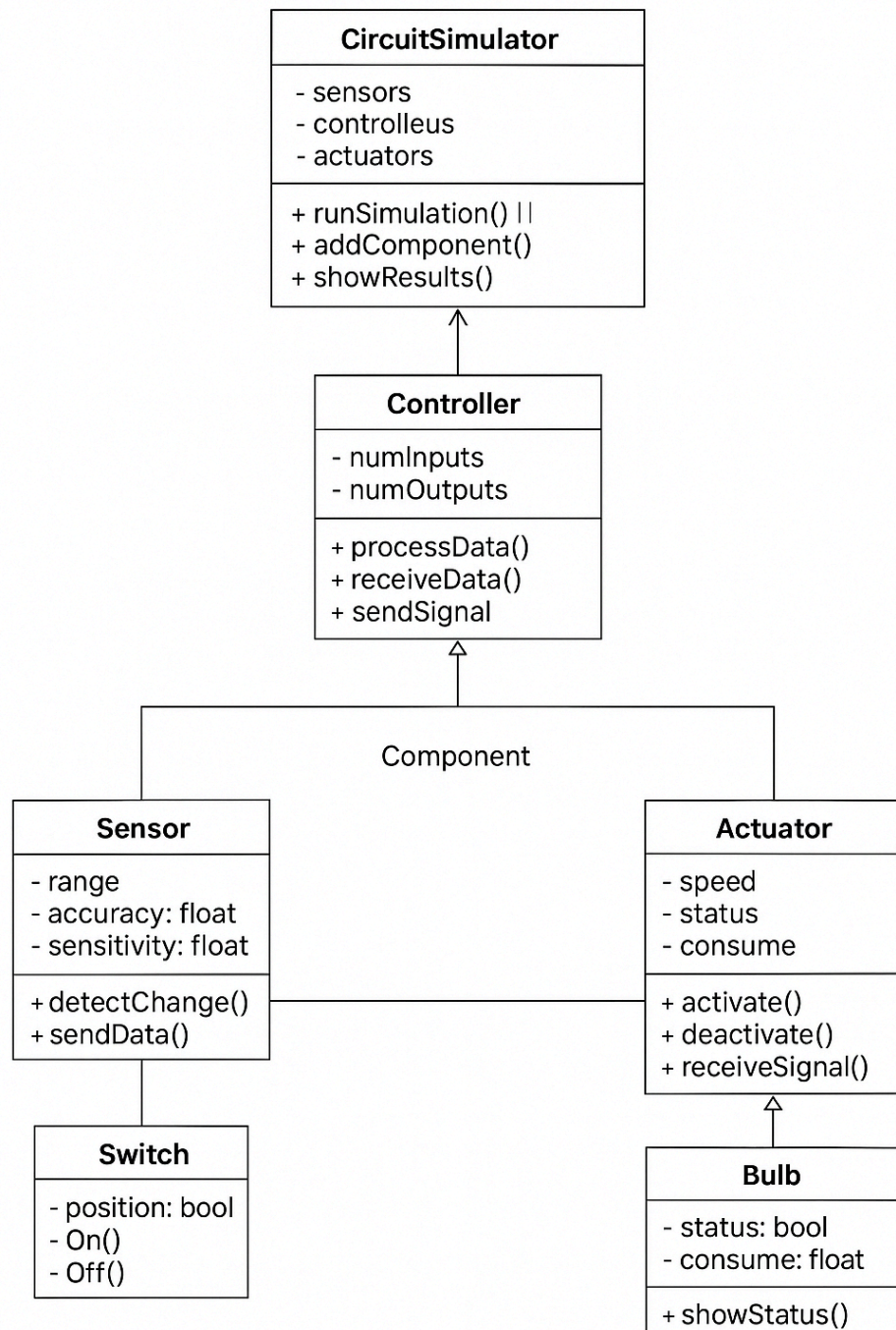
- **Sensor**
- **Actuator** (*base class*)
- **Switch** (*inherits from Actuator*)
- **Bulb** (*inherits from Actuator*)

Responsibilities:

- Represent physical elements of the domotic system
- Expose behavior through well-defined interfaces
- Encapsulate internal attributes such as sensitivity, speed, status, or consumption
- Support polymorphism through inheritance

Interactions:

- Sensors send data to the Controller
- Actuators receive signals from the Controller
- Switch and Bulb override Actuator methods for specific behavior



Documentation and Submission:

Previous Version of the Program (Before Improvements)

- The architecture was not layered: the GUI directly accessed logic classes and the DataManager, creating tight coupling.
-
- The saving system stored only components, not their connections, so loaded projects were incomplete.
-

- There was no factory to recreate logic objects from saved JSON data.
-
- The simulation workflow (controller → UI) was not clearly defined.
-
- UML diagrams and user documentation were incomplete or missing.

Improved Version of the Program

- A Service Layer (SimulationService) was added, creating a clean architecture:
-
- GUI → Service → Logic → Data
-
- Updated DataManager now saves and loads both components and their connections.
-
- A new Factory module (logic/factory.py) recreates objects like Sensor, Bulb, and Switch from JSON.
-
- A consistent simulation workflow was implemented:
- load_project(), save_project(), step(), get_components_state().
-
- Added complete UML diagrams (class + sequence).
-
- Added a User Manual explaining setup, running, and usage.

Implementation Notes

-Applying SOLID Principles

Single Responsibility Principle

Each class handles only one concern:

Sensors detect, controllers process, actuators react.

Open/Closed Principle

New actuators (e.g., fans, alarms) can be added by extending **Actuator**.

Liskov Substitution

Every subclass (Switch, Bulb) can replace Actuator without breaking the system.

Interface Segregation

Classes expose only necessary methods (e.g., actuators only show **activate()** and **deactivate()**).

Usage Instructions

Running the Application

Requirements

Python 3.10+

PyQt5 installed

Run command

`python main.py`

Creating a Circuit

Open the simulator window.

Use the “Add Component” button to create:

Switches

Bulbs

Sensors

Controllers

Components appear visually in the GUI.

Connecting Components

Select “Connect Components”.

Choose source and destination components.

Valid connections are enforced by the business logic:

Sensors → Controller

Controller → Actuator

Invalid connections show an error popup.

Running the Simulation

Press Run Simulation.

If the controller receives an activation signal from a sensor:

Connected actuators change status (e.g., bulbs turn ON).

Results are visually shown on the GUI.

Saving the Circuit

Click “Save Circuit”.

Choose a filename.

A JSON file is generated containing all components and connections.

Loading a Circuit

Click “Load Circuit”.

Select a .json file previously saved.

The GUI reconstructs all components automatically.